



中山大學
SUN YAT-SEN UNIVERSITY

模板Template

中山大学计算机学院



主讲人：万海



wanhai@mail.sysu.edu.cn

目录

CONTENTS

01

泛型编程

02

函数模板

03

类模板

04

非类型模板形参

05

数组的引用



泛型编程 (Generic programming)

泛型编程

独立于任何特定数据类型的编程，这使得**不同类型**的数据
(对象)可以被**相同的代码**操作

在c++中，使用**模板** (*template*) 来进行泛型编程，包括

- 函数模板 (Function template)
- 类模板 (Class template)



泛型编程 (Generic programming)

- 当从**通用代码**创建**实例代码**时，具体数据类型才被确定
- 泛型编程是一种**编译时多态性（静态多态）**。其中，数据类型本身是**参数化**的，因而程序具有多态性特征
- **实例化** (*Instantiation*)：由编译器将通用**模板**代码转换为不同的**实例代码**的过程称为**实例化**

目录

CONTENTS

01

泛型编程

02

函数模板

03

类模板

04

非类型模板形参

05

数组的引用



函数模板 (Function template)

- 相同的处理过程，处理不同类型的数据
 - 减少代码
 - 甚至能处理编程时**未知的**数据类型



函数模板 (Function template)

```
void Swap(int& v1, int& v2)
{
    int temp;
    temp = v1;
    v1 = v2;
    v2 = temp;
}
```

```
void Swap(double& v1, double& v2)
{
    double temp;
    temp = v1;
    v1 = v2;
    v2 = temp;
}
```

```
void Swap(string& v1, string& v2)
{
    string temp;
    temp = v1;
    v1 = v2;
    v2 = temp;
}
```

三个函数除了所处理对象的类型不同之外，代码几乎完全相同。即：三个函数功能类似。



函数模板 (Function template)

//此函数模板可取代上述三函数
//的作用

```
template <typename T>
void Swap( T& v1, T& v2)
{
    T temp;
    temp = v1;
    v1 = v2;
    v2 = temp;
}
```



函数模板的一般形式

```
template < 模板形参表 >
返回值类型 函数名 (形式参数列表)
{
    函数体语句
}
```

不能为空

必须包含模板
形参表中出现
的所有形参

模板形参表形式如下:

typename 模板形参1, **typename** 模板形参2, ...



函数模板 实例化 (instantiation)

```
int main()
{
    std::string s1("rabbit"), s2("bear");
    int iv1 = 3, iv2 = 5;
    double dv1 = 2.8, dv2 = 8.5;

    // 调用函数模板的实例Swap(string&, string&)
    Swap(s1, s2);

    // 调用函数模板的实例Swap(int&, int&)
    Swap(iv1, iv2);

    // 调用Swap的实例Swap(double&, double&)
    Swap(dv1, dv2);
}
```

函数模板的**使用形式**
与普通函数调用相同。

请运行 function-template.cpp

编译是如何完成实例化的？



函数模板 实例化 (instantiation)

调用函数模板的过程：

1. 模板**实参推断** (*template argument deduction*) : 编译器根据函数调用中所给出的**实参的类型**, 确定相应的**模板实参**。
2. 函数模板的**实例化** (*instantiation*) : 模板实参确定之后, 编译器就使用模板实参代替相应的模板形参, 产生并编译函数模板的一个特定版本 (称为函数模板的一个**实例** (instance)) (注意: 此过程中**不进行常规隐式类型转换**) 。

请完成 function-template.cpp 的 EX1



显式（全）模板特化

```
template <typename T>
void Swap( T& v1, T& v2)
{
    T temp;
    temp = v1;
    v1 = v2;
    v2 = temp;
}

template <>
void Swap( int & v1, int & v2)
{
    int temp;
    temp = v1;
    v1 = v2;
    v2 = temp;
    v1 += 10;
    v2 += 10;
}
```

请完成 function-template.cpp 的 EX2

- 在模板当中有些特殊的类型，当想要针对特殊的类型进行一些特殊的操作，这时候就可以用模板的**特化**
- 在正常的模板下面接着编写代码，写一个空的template<>然后写个具体的函数代码来补充
- 如实例所示，当传入的实参类型是int类型，就执行模板的**特化部分**，而非int类型执行正常的**模板推断**



模板重载

对函数模板进行重载：

- 定义名字相同而**函数形参表**不同的**函数模板**
- 或者定义与函数模板同名的**非模板函数(正常函数)**，在其函数体中完成不同的行为

因为可以对函数模板进行重载，那么当进行调用时，编译器是如何确定调用的是这么多同名函数中的哪一个呢？



如何确定调用哪个函数？

函数调用的静态绑定规则（重载协议）：

1. 如果某一同名**非模板函数(指正常的函数)**的形参类型正好与函数调用的实参类型匹配(**完全一致**)，则调用该函数。否则，进入第2步
2. 如果能从同名的**函数模板**实例化一个函数实例，而该函数实例的形参类型正好与函数调用的实参类型匹配(完全一致)，则调用该函数模板的实例函数。否则，进入第3步
 - 在步骤2中：首先匹配函数模板的**特化**，在匹配非指定特殊的函数模板
3. 对函数调用的实参进行**隐式类型转换**后与**非模板函数再次**进行匹配，若能找到匹配的函数则调用该函数。否则，进入第4步
4. 提示编译错误



// 函数模板demoPrint

```
template <typename T>
void demoPrint(const T v1, const T v2){
    cout << "the first version of demoPrint()" << endl;
    cout << "the arguments: " << v1 << " " << v2 << endl;
}
```

// 函数模板demoPrint的指定特殊

```
template <>
void demoPrint(const char v1, const char v2){
    cout << "the specify special of demoPrint()" << endl;
    cout << "the arguments: " << v1 << " " << v2 << endl;
}
```

// 函数模板demoPrint重载的函数模板

```
template <typename T>
void demoPrint(const T v){
    cout << "the second version of demoPrint()" << endl;
    cout << "the argument: " << v << endl;
}
```

// 非函数模板demoPrint

```
void demoPrint(const double v1, const double v2){
    cout << "the nonfunctional template version of demoPrint()" << endl;
    cout << "the arguments: " << v1 << " " << v2 << endl;
}
```



如何确定调用哪个函数?

```
/* 函数调用 */  
string s1("rabbit"), s2("bear");  
char c1('k'), c2('b');  
int iv1 = 3, iv2 = 5;  
double dv1 = 2.8, dv2 = 8.5;
```

```
// 调用第一个函数模板  
demoPrint(iv1, iv2);
```

```
// 调用第一个函数模板的指定特殊  
demoPrint(c1, c2);
```

```
// 调用第二个函数模板  
demoPrint(iv1);
```

```
// 调用非函数模板  
demoPrint(dv1, dv2);
```

```
// 隐式转换后调用非函数模板  
demoPrint(iv1, dv2);
```

```
/* 结果 */  
the first version of demoPrint()  
the arguments: 3 5  
the specify special of demoPrint()  
the arguments: k b  
the second version of demoPrint()  
the argument: 3  
the nonfunctional template version of  
demoPrint()  
the arguments: 2.8 8.5  
the nonfunctional template version of  
demoPrint()  
the arguments: 3 8.5
```

函数模板是不进行隐式转换的，只有非函数模板才进行隐式转换！

目录

CONTENTS

01

泛型编程

02

函数模板

03

类模板

04

非类型模板形参

05

数组的引用



类模板

使用情景：定义可以存放任意类型对象的通用容器类

- 定义一个栈 (stack) 类，即可用于存放int型对象，又可用于存放float、double、string...甚至任意未知类型的元素
- 定义一个队列 (queue) 类，即可用于存放int型对象，又可用于存放float、double、string...甚至任意未知类型的元素

实现方式：为类声明一种**模板**，使得类中的某些数据成员、某些成员函数的参数、某些成员函数的返回值，能取任意类型（包括基本类型和用户自定义类型）



类模板的定义方式

类模板的一般形式:

```
/* 类模板一般形式 */  
template <模板形参表>  
class 类模板名  
{  
    类成员声明  
    ...  
}
```

在类模板外定义成员函数的一般形式:

```
/* 在类模板外定义成员函数的一般形式 */  
template <模板形参表>  
返回值类型 类模板名<模板形参名列表>::函数名(函数形参表)  
{  
    函数实现  
    ...  
}
```

其中模板形参表的形式为: **template** <typename 类型参数1, typename 类型参数2, ...>

(注: 模板形参每项是非类型形参、类型形参、模板形参之一。)



类模板的示例：链表实现的栈类模板

/* 类模板：使用链表实现的栈*/

template <typename **ElementType**>

class Stack {

public:

Stack();

~Stack();

void push(**ElementType** obj)

void pop()

ElementType getTop() **const**

bool isEmpty() **const**;

private:

/* 栈节点类型 */

struct Node

{

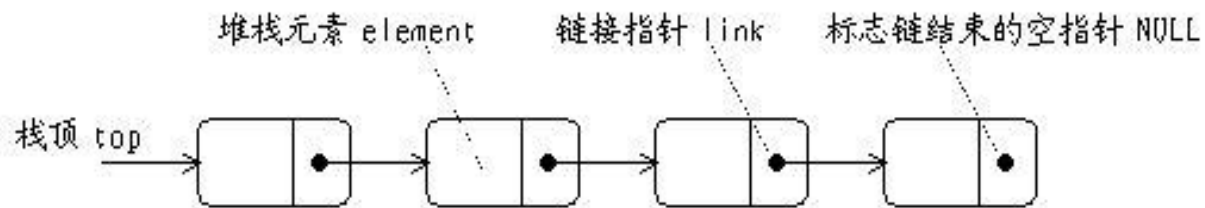
ElementType element;

Node* next;

};

Node* top;

};



throw(std::bad_alloc);

throw(std::logic_error);

throw(std::logic_error);

// 结点中存放的元素
// 指向下一结点的指针

// 栈顶节点



类模板的示例：链表实现的栈类模板

/* 向栈内压入元素 */

template <typename ElementType>

void Stack<ElementType>::push(ElementType obj) throw(std::bad_alloc)

{

Node* temp;

try {

temp = new Node;

temp -> element = obj;

temp -> next = top;

top = temp;

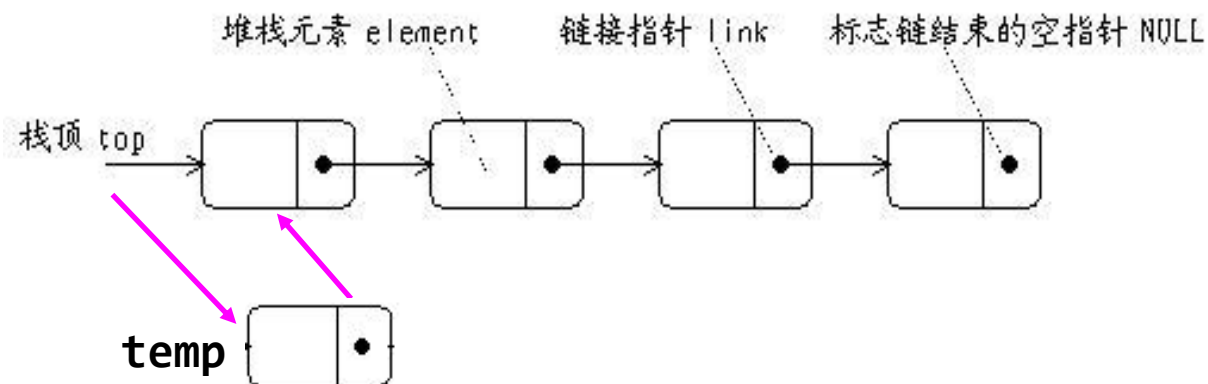
}

catch (std::bad_alloc e) {

throw;

}

}



// 内存分配失败时进行异常处理

// 重新抛出异常



类模板的示例：链表实现的栈类模板

```
/* 从栈顶弹出元素 */  
template <typename ElementType>  
void Stack<ElementType>::pop() throw(std::logic_error)  
{  
    Node* temp;  
    if (top != NULL) {  
        temp = top;  
        top = top -> next;  
        delete temp;  
    }  
    else { // 栈为空时抛出异常  
        throw std::logic_error("pop from empty Stack");  
    }  
}
```



类模板的示例：链表实现的栈类模板

```
/* 获取栈顶元素 */
template <typename ElementType>
ElementType Stack<ElementType>::getTop() const throw(std::logic_error)
{
    if (top != NULL) {
        return top->element;
    }
    else {
        //栈为空时抛出异常
        throw std::logic_error("getTop from empty Stack");
    }
}

/* 判断栈是否为空 */
template <typename ElementType>
bool Stack<ElementType>::isEmpty() const
{
    return top == NULL;
}
```



类模板的示例：链表实现的栈类模板

```
/* 主函数 */
int main()
{
    Stack<int> stack;                // 实例化一个保存int型元素的栈

    for (int i = 1; i < 9; i++)      // 向栈中压入8个元素
        stack.push(i);

    while (!stack.isEmpty()) {       // 栈不为空时循环
        cout << stack.getTop() << " "; // 显示栈顶元素
        stack.pop();                 // 弹出栈顶元素
    }
}
```

```
/* 运行结果 */
8 7 6 5 4 3 2 1
```



类模板的实例化

- 类模板是一个通用类模型，而不是具体类，不能用于创建对象，只有经过**实例化**后才得到具体类，才能用于创建对象
- **实例化**的一般形式：
 - **类模板名 < 模板实参表 >**
- 模板实参是一个实际类型，如int, double等
- 一个类模板可以实例化为多个不同的具体类
 - **Stack<int> stack_int**
 - **Stack<double> stack_double**
 - **Stack<string> stack_string**



类模板的文件组织形式

- 一般而言，调用函数时，编译器只需要看到函数的声明即可。所以可以把函数的声明放在 **.hpp** 文件中，实现在 **.cpp** 的实现文件中，使用函数的地方 **#include** 函数的 **.hpp** 文件即可
- 对于模板则不同，要进行实例化，编译器必须能够访问模板定义的源代码
- 为了在模板中也实现一般的声明定义分离，C++ 提供了两种模板的编译模型



类模板的文件组织形式

- **包含编译模式** (inclusion compilation model)
 - **要求:** 在函数模板或类模板成员函数的调用点, 相应函数的定义对编译器必须是可见的
 - **实现方式:** 在头文件中用**#include**包含实现文件 (也可将模板的实现代码直接放在头文件中)
- **分离编译模式** (separate compilation model)
 - **要求:** 声明和定义分离, 程序员在实现文件中使用保留字**export**告诉编译器, 需要记住哪些模板定义。
 - 不是所有编译器都支持该模式



包含编译模式

//genericStack.hpp

```
#ifndef GSTACK_HPP
```

```
#define GSTACK_HPP
```

类模板的定义和实现代码

```
#endif
```

//client.cpp客户代码

```
#include "genericStack.hpp"
```

```
int main()
```

```
{
```

```
    Stack<int> stack;
```

```
    for (int i = 1; i < 9; i++)
```

```
        stack.push(i);
```

```
}
```

模板的定义和实现都写在 .hpp 文件中。属于包含编译模式

目录

CONTENTS

01

泛型编程

02

函数模板

03

类模板

04

非类型模板形参

05

数组的引用



非类型模板形参

- 两类模板形参：类型模板形参和非类型模板形参
- 非类型模板形参：
 - 相当于模板内部的常量
 - 形式上类似于普通的函数形参
 - 对模板进行实例化时，非类型形参由相应模板实参的值代替
 - 对应的模板实参必须是编译时常量表达式



非模板形参的示例1：以数组实现的栈类模板

```
/* 类模板：使用数组实现的栈*/  
template <typename ElementType, int N>  
class Stack {  
public:  
    Stack();  
    ~Stack();  
    void push(ElementType obj);  
    void pop();  
    ElementType getTop() const;  
    bool isEmpty() const;  
  
private:  
    /* 栈节点类型 */  
    ElementType elements[N];  
    int count;  
};
```



非模板形参的示例1：以数组实现的栈类模板

```
/* 主函数 */
int main()
{
    Stack<int, 10> stack;           // 实例化一个保存int型元素的栈

    for (int i = 1; i < 9; i++)    // 向栈中压入8个元素
        stack.push(i);

    while (!stack.isEmpty()) {     // 栈不为空时循环
        cout << stack.getTop() << " "; // 显示栈顶元素
        stack.pop();              // 弹出栈顶元素
    }
}
```

```
/* 运行结果 */
8 7 6 5 4 3 2 1
```

目录

CONTENTS

01

泛型编程

02

函数模板

03

类模板

04

非类型模板形参

05

数组的引用



函数模板的非模板形参：打印函数

```
/* 非模板形参实例2 */
template <typename T, int N>
void printValues(T (&arr)[N]) {
    for (int i = 0; i != N; ++i)
        cout<< arr[i] << endl;
}

int main()
{
    int intArr[6] = {1, 2, 3, 4, 5, 6};
    double dblArr[4] = {1.2, 2.3, 3.4, 4.5};

    // 生成函数实例printValues(int (&) [6])
    printValues(intArr);
    // 生成函数实例printValues(double (&) [4])
    printValues(dblArr);

    return 0;
}
```



非模板形参的示例2：打印函数

```
/* 非模板形参实例2 */
template <typename T, int N>
void printValues(T (*arr)[N]) {
    for (int i = 0; i != N; ++i)
        cout<< (*arr)[i] << endl;
}

int main()
{
    int intArr[6] = {1, 2, 3, 4, 5, 6};
    double dblArr[4] = {1.2, 2.3, 3.4, 4.5};

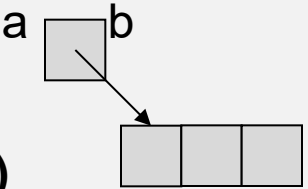
    // 生成函数实例printValues(int (&) [6])
    printValues(&intArr);
    // 生成函数实例printValues(double (&) [4])
    printValues(&dblArr);

    return 0;
}
```

编译器会检查实参数组和形参数组的长度，两长度必须相等。所以这里不仅需要给出长度，而且必须与实参数组长度相等。

```
void f( int (&b)[3] )
{
    int i;
    for( i = 0; i < 3; i++ )
    {
        b[i]++;
        cout << b[i] << endl;
    }
    cout << "address of b in f: " << &b;

    //b++; //错误，b是指针常量
}
```

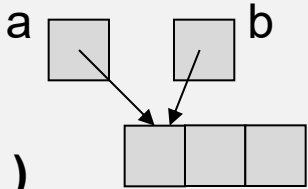


The diagram shows a box labeled 'b' with an arrow pointing to a horizontal array of three boxes, representing an array of 3 elements.

编译器不会检查实参数组和形参数组的长度，所以这里不需要给出长度。如果传递长度，要另外增加形参。

```
void f( int b[ ] )
{
    int i;
    for( i = 0; i < 3; i++ )
    {
        b[i]++;
        cout << b[i] << endl;
    }
    cout << "address of b in f: " << &b;

    b++; //正确，b是指针变量
}
```



The diagram shows a box labeled 'b' with an arrow pointing to a horizontal array of three boxes, representing an array of 3 elements.

```
address of a in main: 0012FF74
2
3
4
address of b in f: 0012FF74
```

```
int main()
{
    int a[3] = {1,2,3};
    cout << "address of a in main: "<< &a ;
    f(a);
}
```

```
address of a in main: 0012FF74
2
3
4
address of b in f: 0012FF24
```



不使用非模板形参实现

```
/* 不使用非模板形参实现 */
template <typename T>
void printValues(T* arr, int N) {
    for (int i =0; i != N; ++i)
        cout<< arr[i] << endl;
}

int main()
{
    int intArr[6] = {1, 2, 3, 4, 5, 6};
    double dblArr[4] = {1.2, 2.3, 3.4, 4.5};

    // 生成函数实例printValues(int*, 6)
    printValues(intArr, 6);
    // 生成函数实例printValues(double*, 4)
    printValues(dblArr, 4);

    return 0;
}
```



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



中山大学MOOC课程组